



PlugFrame Presentation



Summary

1 PlugFrame Project Overview.....	4
1.1 Introduction.....	4
1.2 Purpose of PlugFrame.....	4
1.3 Scalability and Reuse.....	4
2 PlugFrame Bundles Overview.....	6
2.1 Architecture Overview.....	6
2.2 Roles of the Main Bundles.....	6
2.2.1 Framework.....	6
2.2.2 Logger.....	6
2.2.3 Console & Display.....	6
2.2.4 GuiConsole & GuiDisplay.....	7
2.2.5 Users.....	7
2.3 Interaction Model.....	7
2.4 Design Vision.....	7
3 PlugFrame's Library & Plugins.....	9
3.1 Diagram Purpose.....	9
3.2 Deployment Elements.....	9
3.2.1 core-lib (PlugFrame Core Library).....	9
3.2.2 Bundles as Dynamic Plugins.....	9
3.3 Three Types of Launchers (planned).....	10
3.3.1 Headless Launcher.....	10
3.3.2 GUI Launcher (Qt Widgets).....	10
3.3.3 Future QML Launcher (planned).....	10
3.4 Deployment Flexibility.....	10
3.5 Design Intent.....	11
4 PlugFrame Core Bundles Classes.....	12
4.1 Diagram Purpose.....	12
4.2 Core Structure.....	12
4.3 Interfaces and Responsibilities.....	12
4.4 Design Objective.....	13
5 PlugFrame Bundle Lifecycle.....	15
5.1 Key Lifecycle States.....	15

5.1.1 Lifecycle Limitation (Current Version).....	16
6 Bundle Initialization – Sequence Diagram.....	17
6.1 What This Diagram Shows.....	17
6.2 Technical Insights.....	17
6.3 Target Audience.....	17
7 Conclusion & Next Steps.....	19

1 PlugFrame Project Overview

1.1 Introduction

This diagram provides a conceptual snapshot of the PlugFrame project. It is meant to introduce the motivation behind the framework, its envisioned applications, and its foundational philosophy. It does not focus on technical internals, but rather outlines what PlugFrame aims to enable and how it fits into broader software challenges.

1.2 Purpose of PlugFrame

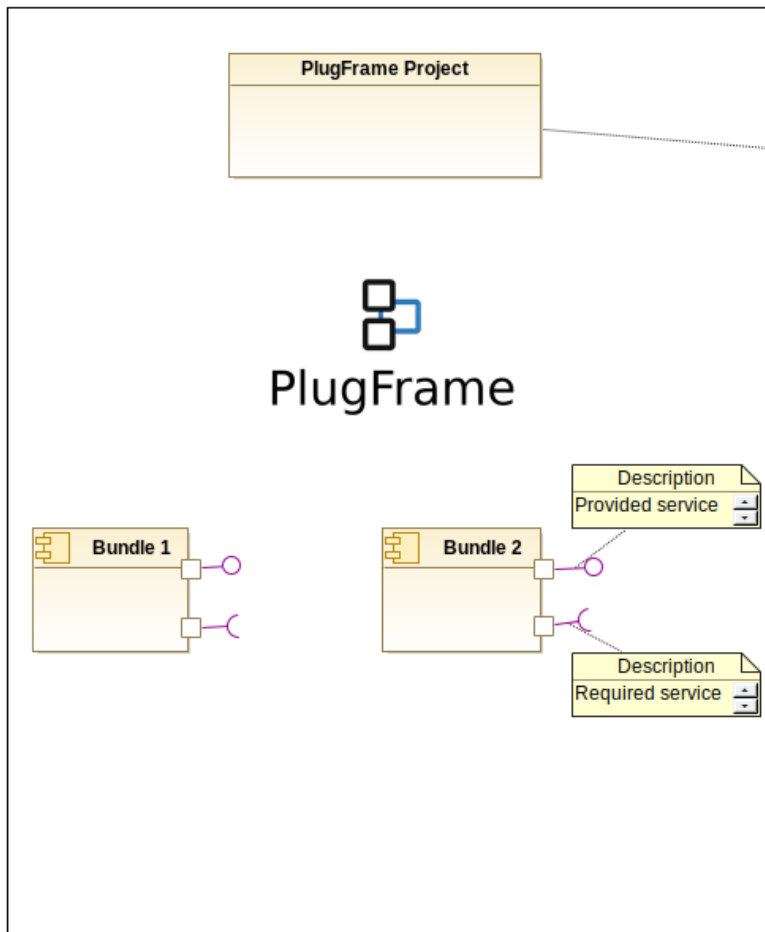
PlugFrame was created to serve as a foundational framework for modular software systems, particularly in domains requiring supervision, control, and integration of heterogeneous equipment. Inspired by the principles of OSGi but tailored to C++ using the Qt plugin system, PlugFrame aims to deliver an extensible runtime platform where application logic is divided into loosely coupled, independently deployable modules known as *bundles*. This architecture fosters better separation of concerns, long-term maintainability, and component reuse across projects.

1.3 Scalability and Reuse

The framework is designed with scalability in mind. Developers can extend applications by simply adding new bundles, without modifying the core platform. Reuse is a key goal: bundles developed for one application can be shared across others, and core services like logging, configuration, or user management can be standardized. This approach enables faster development cycles and more robust, evolvable software ecosystems.



Licence : GNU GPL v3
(c) 2026 Christophe Mars / ELEKDOM



Description

== PlugFrame Project Overview ==

PlugFrame is a modular C++ framework directly inspired by the fundamental principles of the OSGi (Open Services Gateway Initiative) architecture.

The goal is to provide a lightweight, scalable, and reusable platform capable of supporting complex and heterogeneous applications, while leveraging Qt plugins to dynamically manage software extensions.

== Objectives ==

- > Reproduce the OSGi approach in the C++/Qt world: modularity, encapsulation, and component lifecycle management.
- > Provide a technical foundation structured around independent bundles (loaded at startup in this first version).
- > Enable easy reuse of components, strong decoupling, and increased maintainability of application projects.
- > Propose an extensible architecture to meet the growing needs of business applications, while maintaining a lightweight footprint.

== Design Principles ==

- > Use of Qt plugins (QPluginLoader) to simulate the OSGi-like bundle loading mechanism.
- > Implementation of a centralized service registry, ensuring discovery and interfacing between components (exported/imported services).
- > Clear separation between the platform (core) and application bundles.
- > Orientation toward a hexagonal architecture facilitating testing, integration, and functional evolution.

== Project Status ==

This first version constitutes a functional base, designed to evolve gradually based on:

- > needs identified during the development of concrete applications (such as Oplink);
- > future technical constraints (support for dynamic hot loading, etc.).

== Outlook ==

- > Integration of a shared configuration manager.
- > Implementation of a richer bundle lifecycle (activation/deactivation).
- > Dynamic service discovery
- > Openness to an open source community, facilitating adoption and contributions.

2 PlugFrame Bundles Overview

2.1 Architecture Overview

This diagram presents the modular organization of PlugFrame through its core *bundles*. Each bundle is implemented as a dynamic library and communicates with others through the service registry provided by the Framework. This structure promotes separation of concerns, extensibility, and ease of maintenance.

2.2 Roles of the Main Bundles

2.2.1 Framework

Acts as the execution engine of the platform. It is responsible for loading bundles at startup, managing their lifecycle (initialization and startup), and orchestrating service registration and discovery. All runtime operations hinge on this foundational component.

2.2.2 Logger

Provides centralized logging capabilities using Qt's logging mechanisms (qDebug, qWarning, etc.). It exposes a log service used by other bundles to record messages in a unified and structured way. This service can be extended or redirected (e.g., to a file or GUI display) in future versions.

2.2.3 Console & Display

These two bundles form the **text-based interaction layer** of the platform:

- **Console** handles **command input** in headless (non-GUI) contexts. It currently supports three core commands:
 1. List bundles in the **Initialized** state (for diagnostics).
 2. List **Started** bundles (for diagnostics).
 3. **Quit** the platform cleanly (e.g., to shut down a headless server).
- **Display** is in charge of **rendering output** in text mode. It synchronizes access to the screen (typically `stdout`).

2.2.4 GuiConsole & GuiDisplay

These two bundles mirror the behavior of Console and Display, but in a **graphical context** using Qt:

- **GuiConsole** integrates a GUI-based command input system into the application's main window. It implements a view/controller for the same three commands as the Console bundle.
- **GuiDisplay** manages the application's main window (`MainWindow`) and overall graphical layout. It also exposes a `GuiBuilderServiceInterface` service, enabling other GUI-capable bundles to register and inject their visual components into the shared interface.

2.2.5 Users

Implements authentication logic. This bundle was initially developed for OpLink, where client applications require login verification. Given its general-purpose utility, it has been incorporated into PlugFrame. It provides a service interface for managing user credentials and authentication workflows, but is only relevant when applications require such capabilities.

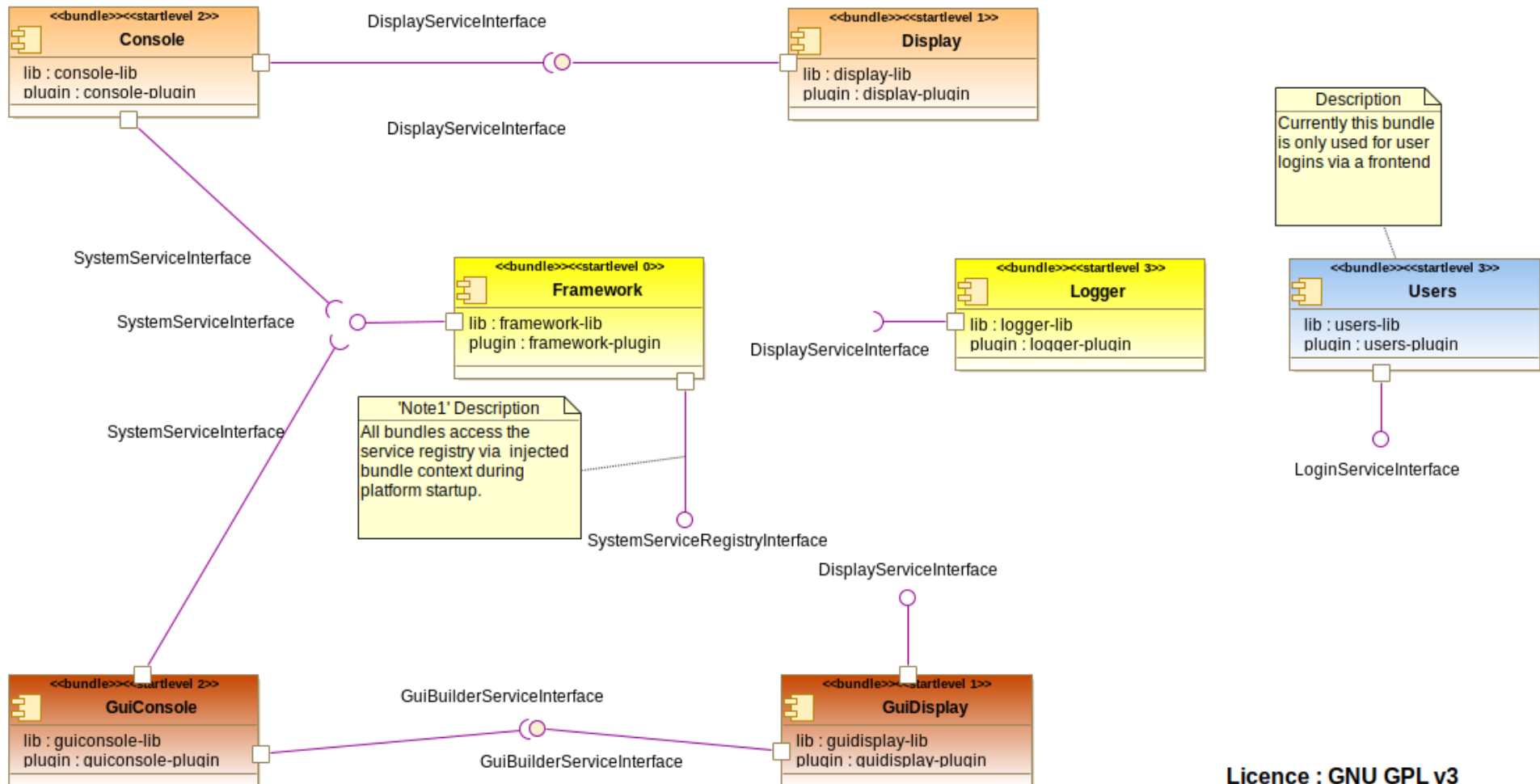
2.3 Interaction Model

All bundles interact through **provided and required services**, registered and resolved via the Framework's service registry. This ensures maximum decoupling and encourages modularity, making it easy to add, replace, or remove functionalities.

2.4 Design Vision

This architecture supports headless operation (via Console/Display) and rich graphical interfaces (via GuiConsole/GuiDisplay), making PlugFrame adaptable to a wide range of contexts—from lightweight IoT platforms to desktop applications with full GUI. All components are designed with reusability and interoperability in mind, forming a robust foundation for future PlugFrame-based applications.

Note, the console and display bundles are to be used in a command-line terminal application context



Note :the guiconsole and guidisplay bundles are to be used in a Gui application context

Licence : GNU GPL v3
(c) 2026 Christophe Mars / ELEKDOM

3 PlugFrame's Library & Plugins

3.1 Diagram Purpose

This diagram illustrates the deployment structure of PlugFrame from a system-level perspective. It identifies the two types of **application launchers**, the core **dynamic library (core-lib)**, and the set of dynamically loaded **bundle plugins** that compose the running platform.

3.2 Deployment Elements

3.2.1 core-lib (PlugFrame Core Library)

This is the essential shared library loaded by both types of launchers. It encapsulates all the **fundamental abstractions**, **interfaces**, and **service contracts** of PlugFrame. It defines:

- Bundle metadata and lifecycle contracts,
- Service registration and discovery mechanisms,
- Abstract interfaces for core services,
- The technical foundation upon which every plugin and application is built.

All bundles dynamically link against core-lib and use its facilities to interact with the platform.

3.2.2 Bundles as Dynamic Plugins

Each bundle (e.g., Framework, Logger, Console, GuiDisplay...) is implemented as a **Qt plugin** (a .so or .dll file). These are dynamically discovered and loaded by the Framework at platform startup. This plugin model supports:

- Decoupled deployment,
- Feature isolation,
- Easy extensibility (just drop a new plugin in the correct folder),
- Clean separation of responsibilities.

Each plugin must implement the standard `IBundle` interface and expose metadata (name, version, services, dependencies) understood by the Framework.

3.3 Three Types of Launchers (planned)

3.3.1 Headless Launcher

Starts the PlugFrame platform in **non-GUI mode**, ideal for lightweight services, daemons, or server-side logic. It typically loads text-based bundles such as Console and Display.

3.3.2 GUI Launcher (Qt Widgets)

Integrates full **Qt GUI support** and starts the platform with interactive bundles like GuiConsole and GuiDisplay, suitable for desktop or embedded graphical applications.

3.3.3 Future QML Launcher (planned)

A new launcher is envisioned to support **QML-based applications**, leveraging the declarative power of QtQuick. This would enable building modern, dynamic UIs while maintaining PlugFrame's modular backend. This launcher would integrate seamlessly with the platform's plugin system and GUI services.

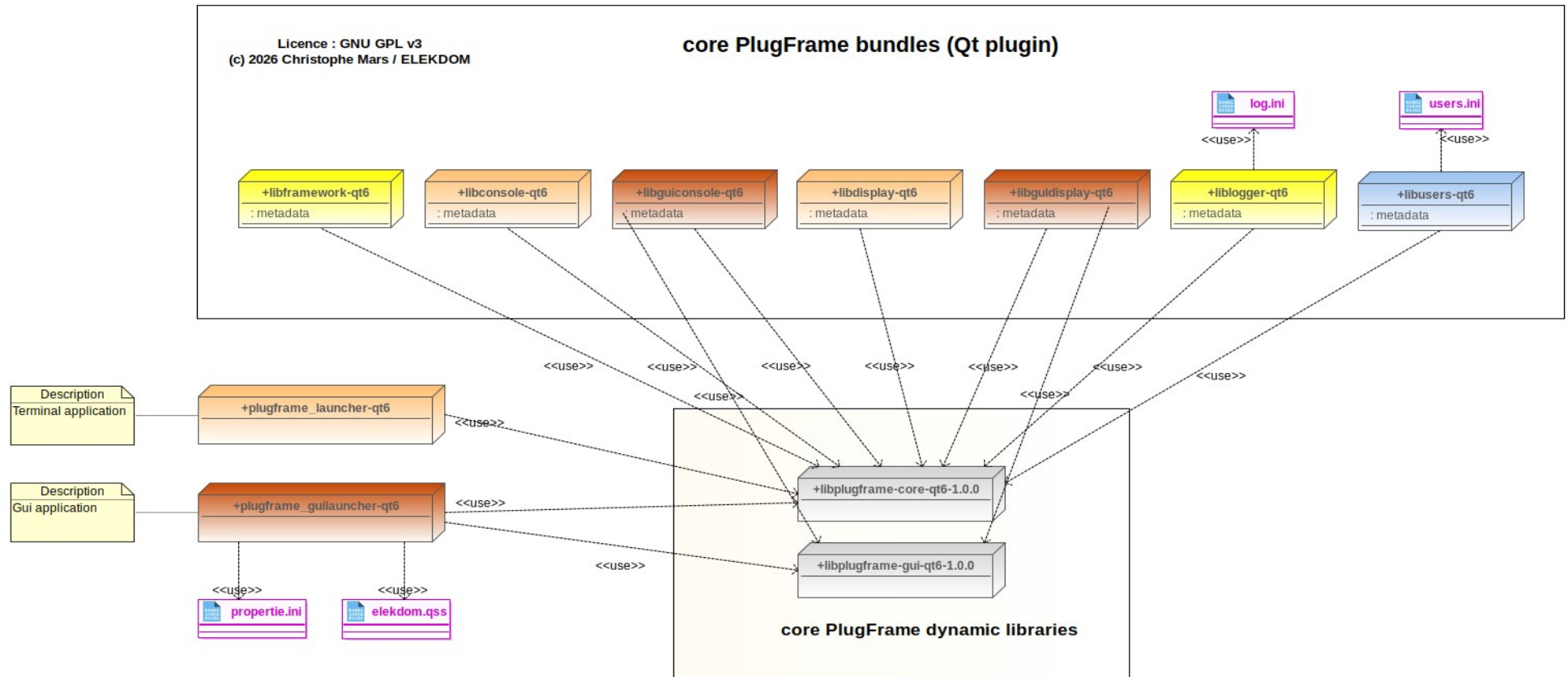
3.4 Deployment Flexibility

The separation between the launcher and the plugins makes PlugFrame extremely adaptable:

- New bundles can be developed and deployed independently,
- GUI or headless behavior can be selected without recompiling the platform,
- Platform configurations are extensible at runtime without altering the core logic.

3.5 Design Intent

This modular deployment model reflects the architectural values of PlugFrame: **scalability**, **reuse**, and **loose coupling**. It enables a consistent execution framework across a wide range of environments (from CLI tools to full-featured GUI apps), while maintaining a minimal and maintainable footprint.



4 PlugFrame Core Bundles Classes

4.1 Diagram Purpose

This class diagram offers a structural view of how a bundle is built within the PlugFrame framework. It is primarily intended for a **technical audience** — such as software architects, system designers, or developers — who require insight into how PlugFrame decouples plugin infrastructure from functional logic.

4.2 Core Structure

At the heart of each bundle is a Qt dynamic plugin. PlugFrame standardizes plugin loading by requiring an `IPlugin(BundleInterface)` interface implementation. This interface acts as the **entry point** into the plugin and exposes metadata and factory logic to the framework.

A critical design aspect is the **explicit separation** between:

- the **Plugin Layer**, which is Qt-dependent and exposes the bundle to the platform, and
- the **Bundle Implementation**, which conforms to the `IBundle(Bundle4PluginInterface)` interface and holds the actual business logic.

The `createImplementation()` method in the plugin serves as a factory, enabling the framework to instantiate the functional part of the bundle independently from the Qt plugin mechanism. This makes **injection and testing** more straightforward.

4.3 Interfaces and Responsibilities

- **IBundle**: Defines lifecycle methods (`init`, `start`, `stop`) and represents the core executable logic of a bundle.
- **IPlugin**: Used by PlugFrame to discover and load plugins, and to instantiate the actual `IBundle`.
- **IBundleContext**: Injected at runtime, it enables bundles to **interact with the service registry** — registering or retrieving services.

4.4 Design Objective

This architecture enforces **clean modularity**, simplifies **maintainability**, and supports **future extension**. Qt plugin mechanics are strictly isolated, while core logic interacts only with PlugFrame-defined interfaces. This helps ensure long-term sustainability and integration consistency across the platform.

Licence : GNU GPL v3
(c) 2026 Christophe Mars / ELEKDOM

'Bundle's Parts' Description

A bundle is defined by three main parts:

1. The elements that allow its construction, consisting of a factory and a builder. This part will be specialized based on the specifics of the real bundle.
2. The elements for its interaction with other bundles. In particular, the Listener and the Event Emitter. These elements will eventually be specialized according to the particularity of the real bundle.
3. Its implementation must be derived from BundleImplementation or LongstartBundleImplementation. In addition, bundles that provide services must implement them in child classes of ServiceImplementation.

BundleBuilder

- + build()
- + getBundle()
- + getImplementation()
- + getLogBundleName()
- + getFactory()
- + setHeaders()
- + loadManifestHeaders()
- + setEmitter()
- + setListener()
- + defaultListening()
- + addExportedService()
- + specificBulk()
- buildExportedServices()

BundleBuilderInterface

- + getBundle()
- + getImplementation()
- + getLogBundleName()
- + getFactory()
- + setHeaders()
- + loadManifestHeaders()
- + setEmitter()
- + setListener()
- + defaultListening()
- + addExportedService()

Loggable

BundlePluginInterface

- + setPlugin()
- + setFileName()
- + setAbsolutePath()
- + setMetadata()
- + init()
- + getServiceImplementation()
- + start()
- + stop()
- + getFileName()
- + getTmp()
- + getStartLevel()
- + getState()

Description

A bundle has information transmitted through Qt plugin metadata. This metadata are declared in a json file included by Qt in the plugin library.

BundleHeaders

- + splitNameVersionFromServiceDeclaration()
- + insertHeader()
- + getHeader()
- + getStartLevel()
- + getTmp()
- + getConfFile()
- + getProvidedServices()
- + getRequiredServices()

BundleFactory

- + createBuilder()
- + createHeaders()
- + createBundleEmitter()
- + createBundleListener()
- + createServiceImplementation()
- + createBundleStartingEvent()
- + createBundleStartedEvent()
- + createBundleStoppingEvent()
- + createBundleStoppedEvent()

DefaultBundleFactory

- + createServiceImplementation()

ServiceImplementationInterface

- + exported services
- + serviceName : string

ServiceImplementation

Bundle

- + setPlugin()
- + setFileName()
- + setAbsolutePath()
- + setMetadata()
- + getPlugin()
- + init()
- + start()
- + stop()
- + getHeader()
- + getTmp()
- + getStartLevel()
- + getFactory()
- + setHeaders()
- + loadManifestHeaders()
- + setEmitter()
- + getEmitter()
- + setListener()
- + defaultListening()
- + setListener()
- + defaultListening()
- + createFactory()
- + pure virtual++ # start()
- + pure virtual++ # stop()

BundleImplementation

- + addExportedService()
- + getServiceImplementation()
- + start()
- + stop()
- + pure virtual++ # qServiceInterface()
- + registerService()
- + registerExportedServices()

LongstartBundleImplementation

- + started()
- + start()
- + defaultListening()

BundleObject

- + getBundle()
- + getBundleId()
- + getFactory()
- + getLogBundleName()

BundleEmitter

- + registerListener()
- + unregisterListener()
- + postBundleStartingEvent()
- + postBundleStartedEvent()
- + postBundleStoppingEvent()
- + postBundleStoppedEvent()

BundleListener

- + onEvent()

BundleListener4ios

- + onEvent()
- + pure virtual++ # onFrameworkStartedEvent()

BundleInterface

- + setFileName()
- + setAbsolutePath()
- + setMetadata()
- + setPlugin()
- + init()
- + start()
- + stop()
- + getHeader()
- + getTmp()
- + getStartLevel()
- + getState()
- + state2String()
- + BundleState

PlugFrame Bundle' Description

A bundle is physically a Qt plugin (currently the Q5 API). Refer to the Qt documentation for a description of this API. PlugFrame defines a decoupling between the implementation of a bundle and the interface with the Qt plugin API.

Plugin

- + setFileName()
- + setAbsolutePath()
- + setMetadata()
- + init()
- + start()
- + stop()
- + pure virtual++ # createImplementation()
- + pure virtual++ # bindServicesImplementations()
- + getName()
- + getTmp()
- + getStartLevel()
- + getState()
- + state2String()
- + _init()

ServiceInterface

RealServiceInterface

- + method1()
- + method2()

RealPlugin

- + createImplementation()
- + bindServicesImplementations()
- + method1()
- + method2()

How to define Bundle' Description

To create a functional bundle:

- Define its implementation section, based on the bundle section of the unit design.
- Define its "plugin with Qt" section by implementing a "RealPlugin" class and optionally its "RealServiceInterface" service interface.

5 PlugFrame Bundle Lifecycle

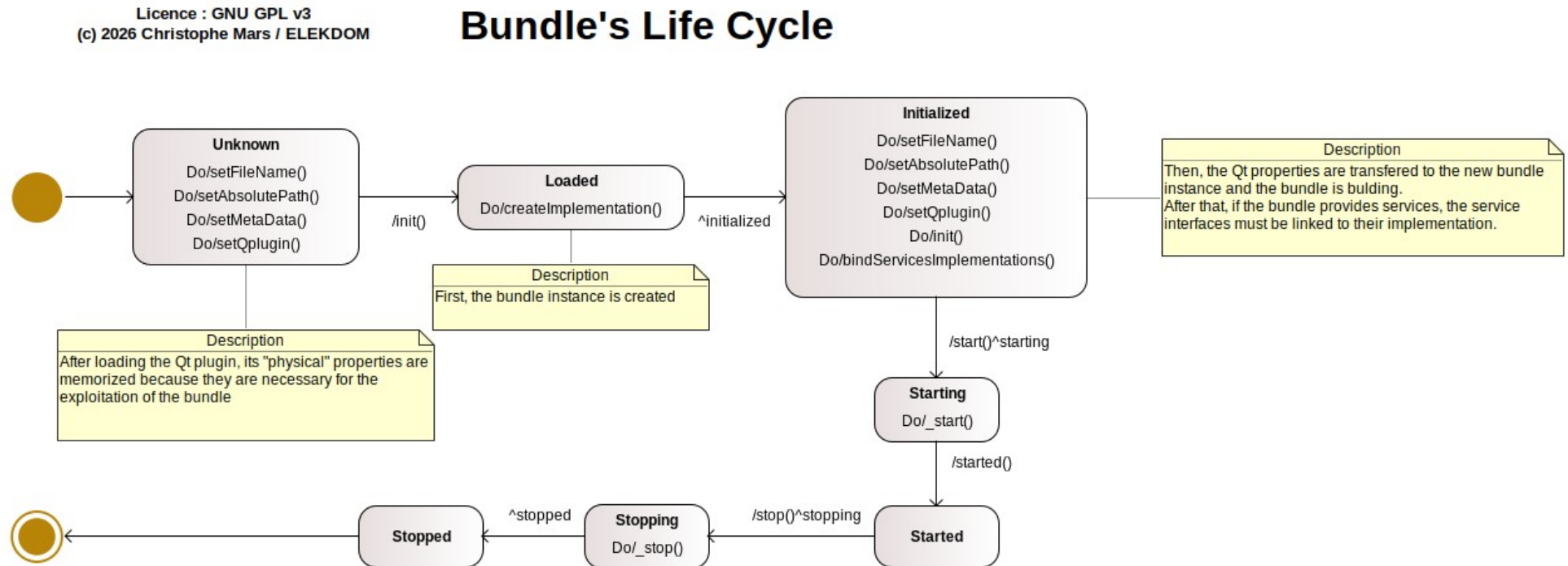
This state diagram outlines the **execution lifecycle** of a PlugFrame bundle, from its instantiation to its final deactivation. It provides a high-level understanding of how the framework manages modular components over time. The diagram is particularly relevant for **technical audiences** such as developers and system integrators, as it clarifies the expected behavior and state transitions of each bundle.

5.1 Key Lifecycle States

1. **Unknown:** This state represents a plugin that has been successfully loaded by the Framework, but which has not yet been initialized as a functional bundle within the PlugFrame platform.
2. **Loaded:** After the plugin has been loaded, the platform initiates its initialization in two phases. The **Loaded** state corresponds to the first phase, in which the platform creates the concrete implementation of the bundle via the plugin's factory mechanism. At this stage, the bundle exists as an instantiated component but is not yet context-aware.
3. **Initialized:** This state marks the completion of the second phase of the initialization process. After the implementation is instantiated (Loaded), the bundle is fully constructed with its internal structure and service declarations. At this stage, services are not yet active. The bundle is now fully prepared to be started by the platform.
4. **Starting:** In this transitional state, the platform initiates the startup of the bundle according to its declared startup level, as specified in its metadata headers. The platform calls the `start()` method, which serves as the official entry point for bundle activation. During this phase, the bundle typically registers and activates its exported services, making them available to other components within the platform.
5. **Started:** The bundle is now fully operational. All services have been registered and are available for use by other components. The bundle is running and can interact with the platform and other bundles as needed.
6. **Stopping:** The platform is currently shutting down the bundle. This transitional state involves invoking the bundle's `stop()` method, allowing it to gracefully release resources, unregister services, and perform any necessary cleanup.
7. **Stopped:** The bundle has completed its shutdown process. All associated services are unregistered, and the bundle is now inactive. It remains loaded but can no longer interact with the platform.

5.1.1 Lifecycle Limitation (Current Version)

Currently, PlugFrame only manages the lifecycle of bundles during the platform's startup and shutdown. It is not yet possible to start or stop individual bundles dynamically at runtime, unlike fully dynamic frameworks like OSGi.



The "framework" bundle is responsible for the bundle lifecycle.

The "framework" is started by the application via the launcher.

6 Bundle Initialization – Sequence Diagram

This sequence diagram illustrates the internal process used by the PlugFrame platform to initialize a bundle after its plugin has been loaded.

6.1 What This Diagram Shows

Once the Framework loads a plugin, it performs a two-step initialization:

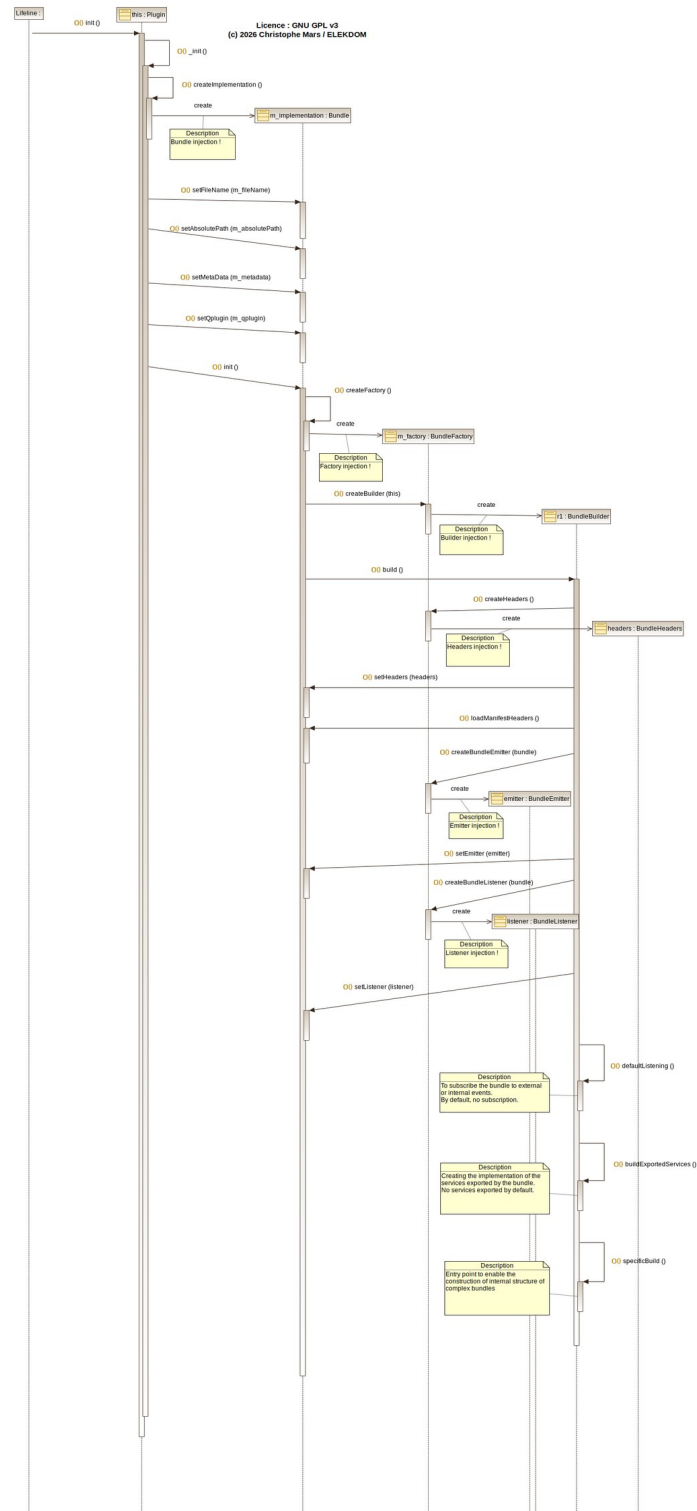
1. **Plugin Instantiation** : The platform uses Qt’s plugin system to load the dynamic library and obtain a reference to the plugin.
2. **Bundle Injection** via `createImplementation()`: The plugin creates the actual implementation object (derived from `IBundle`). This pattern decouples Qt mechanics from the business logic and allows the platform to manage all bundles through a uniform interface.

6.2 Technical Insights

- `createImplementation()` is a key method called by the Framework.
- It enables *inversion of control* by allowing the bundle to define its structure and behavior independently.
- The resulting `IBundle` instance is then passed through further initialization (e.g., setting its context and registering metadata).

6.3 Target Audience

This diagram is intended for developers and architects who want to understand how PlugFrame instantiates and prepares modular bundles for integration within the platform. It reflects the internal control flow handled by the Framework at platform startup.



7 Conclusion & Next Steps

This document provides a high-level technical overview of the PlugFrame platform — a modular and extensible software foundation inspired by OSGi principles and adapted to the Qt/C++ ecosystem. Through this initial series of diagrams, the reader gains insight into the architectural vision, bundle system, and foundational lifecycle mechanisms that power PlugFrame.

Although a demonstrable and functional version of the platform already exists, the project is still in active development. The current version continues to evolve as new needs arise, especially in the context of its integration with Oplink — a real-world application built on top of PlugFrame.

Open Source Availability

The source code for PlugFrame is **released on GitHub** : <https://github.com/ELEKDOM/PlugFrame>.

Interested? Let's Collaborate

PlugFrame is more than an internal framework — it is intended to become a robust technical foundation for future developments in smart systems, supervision platforms, and modular control-command solutions.

If you are:

- A **developer** or **architect** looking for a solid modular base in C++/Qt,
- An **organization** or **startup** needing a custom technical architecture for supervision or automation,
- Or simply interested in learning more about the approach and contributing,

 **Feel free to reach out** for collaborations, custom developments, training, or technical consulting via ELEKDOM.

Let's turn innovation into something sustainable and shared.